

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 1.3]

PROCESS INTERACTION AND COORDINATION

BASIC MECHANISMS & ABSTRACTION FOR PROCESS COORDINATION

- The algorithms for the CS problem described in previous module can be run on a *bare* machine
 - they use only machine language instructions that the computer provides
 - too low level to be used efficiently and reliably
- > introduction of basic mechanisms and abstractions for process coordination
 - provided directly by the concurrent machine and used in concurrent languages
- Main constructs
 - **semaphores**
 - **monitors**

SEMAPHORES

Per risolvere qualsiasi problema di coordinamento ed interazione tra processi

- Introduced by Dijkstra in 1968, semaphores are a very simple but powerful ~~general-purpose construct~~^{meccanismo} which makes it possible to solve almost *any* mutual exclusion and synchronization problem
 - informally, a semaphore functions as street semaphore, *blocking* and *unblocking* process execution (car movement) according to the need
- Semaphore as a primitive data type provided by the concurrent machine

I lock non sono general-purpose come i semafori:
si possono usare solo per Mutua esclusione (es:
non si usano per fare sincronizzazione)

SEMAPHORE DATA TYPE

- A semaphore S is a compound data type with two fields:
 - $S.V$ is an *integer* ≥ 0
 - $S.L$ is a **set** of process (id) insieme di processi bloccati in attesa di poter svolgere l'operazione sul semaforo
- It can be initialized with:
 - a value $k \geq 0$ for $S.V$ Valore Iniziale del semaforo
 - the empty set $\{\}$ for $S.L$
 - e.g. semaphore $S = (k, \{\})$
- It provides two basic *atomic* operations
 - **wait(S)**
 - also called **P(S)** from Dijkstra original choice
 - **signal(S)**
 - also called **V(S)** from Dijkstra original choice

WAIT OPERATION

- Definition (p is current process executing wait):

<> indica che le istruzioni dentro sono eseguite insieme in modo atomico

```
wait(S)=  
< if (S.V > 0)  
    S.V ← S.V - 1  
else  
    S.L = S.L + {p}  
    p.state ← blocked >
```

insieme dei processi
che hanno fatto una
wait e si sono bloccati

- Description
 - if the value of the semaphore V is > 0 (~the semaphore is *green*), then it is simply decremented.
 - otherwise if the value V = 0 (the semaphore is red), then the process is blocked
 - *p is blocked on the semaphore S*
- Note that wait is meant to be *atomic*

SIGNAL OPERATION

- Definition:

<> indica che le istruzioni dentro sono eseguite insieme in modo atomico

```
signal(S)=  
< if (S.L = {})  
    S.V ← S.V + 1  
else  
    let q ← arbitrary element of S.L  
    S.L ← S.L - {q}  
    q.state ← ready >
```

Se l'insieme è vuoto, incremento

Se l'insieme non è vuoto, scelgo un processo tra quelli in S.L e lo sblocco, senza modificare S.V

- Description

- If no process is waiting, then the semaphore value is incremented
- otherwise select a process q blocked on the semaphore, and unblock it.

- Also `signal` is meant to be *atomic*

SEMAPHORE INVARIANT

- Let k be the initial value of the integer component of the semaphore, $\#signal(S)$ the number of $signal(S)$ statements that have been executed, and $\#wait(S)$ the number of $wait(S)$ statements that have been completely executed.
 - a process that is blocked when executing $wait(S)$ is not considered to have successfully executed the statement
- THEOREM** Si può dimostrare tramite induzione
 - A semaphore S satisfies the following invariants:

1 $S.V \geq 0$ il semaforo ha sempre valore ≥ 0 perchè le wait non decrementano se $S.V$ è 0

2 $S.V = k + \#signal(S) - \#wait(S)$

Cosa succederebbe se contassimo anche le wait dei processi bloccati? I conti non tornerebbero più e otterremmo valori impossibili (negativi) per il semaforo.

se $S.V$ uguale ad insieme vuoto, incrementa

se $S.V > 0$, decrementa $S.V$

TYPES OF SEMAPHORES

Essendo che i semafori possono essere usati per fare cose diverse, esistono 3 macro categorie

→ Semafori usati per implementare Mutua Esclusione

• **Mutex or binary semaphores**

- semaphores whose integer component can take only two values, 0 and 1

S.V init. ad 1

S.V rappresenta quanti permessi sono disponibili per entrare in sezione critica (cioè quanti processi possono andare in CS, cioè solo 1 alla volta)

- the name derives from their typical use for implementing mutual exclusion

- wait: prendo la risorsa/permesso se disponibile, se no vado in coda
- signal: rilascio la risorsa/permesso e, se ho un processo in coda, sveglio il processo così che possa prendere la risorsa/permesso rilasciato

→ Gestione di competizione tra un insieme di permessi/risorse

• **General or counting or resource semaphores**

Semaforo rappresenta il numero di risorse/permessi a disposizione

- semaphores whose integer component can take any value ≥ 0

S.V init. alle quantità di risorse/permessi disponibili

- wait: prendo la risorsa/permesso se disponibile, se no vado in coda
- signal: rilascio la risorsa/permesso e, se ho un processo in coda, sveglio il processo così che possa prendere la risorsa/permesso rilasciato

• **Event semaphores**

→ Gestione della sincronizzazione delle azioni tra processi (cooperazione tra processi)

- initialised with 0, used for synchronisation purpose

→ si parla di eventi che qualcuno aspetta o qualcuno deve segnalare

→ processo che fa wait su semaforo che vale 0, si blocca a priori (non è ancora capitato l'evento)
processo segnala che l'evento è successo tramite signal

Implementazione di wait e signal rimangono le stesse

DEFINITIONS OF SEMAPHORES

- There are several different definitions of the semaphore type

— differences relate to the specification of liveness properties, and do not affect the safety properties that follow from the semaphore invariants

derivano dagli

- Main types

- A — **strong vs weak** semaphores
- B — **busy-wait** semaphores

Non importa quale variante di semaforo tu scelga (strong, weak o busy-wait): la proprietà di safety del tuo algoritmo (es. il fatto che due processi non entrino mai contemporaneamente in sezione critica) è sempre garantita matematicamente dall'invariante del semaforo. Ciò che cambia tra le varie definizioni è unicamente la liveness (ovvero la garanzia che un processo non rimanga bloccato ad aspettare per sempre, andando in starvation).

A

STRONG SEMAPHORES

In un insieme, non esiste un ordine tra gli elementi di un insieme, cosa che si ha nelle liste e code, dove l'elemento viene aggiunto in fondo

- In **strong** semaphore **S.L** is not a set, but a **queue** semaphores in which **S.L** is a **set** are called **weak**

e questo può causare starvation perchè essendo un set, durante la signal, un processo (se ne ho 3+ processi in competizione) può non essere mai risvegliato

signal, se uso semafori strong, risveglia il primo della lista/coda

```
wait(S) =
< if (S.V > 0)
    S.V ← S.V - 1
else
    append(S.L, p)
    p.state ← blocked >
```

```
signal(S) =
< if (S.L = empty_queue)
    S.V ← S.V + 1
else
    let q ← take(S.L)
    q.state ← ready >
```

- Important property: **no starvation**
 - for a strong semaphore *starvation is impossible for any number N of processes*

B

BUSY-WAIT SEMAPHORES

- Semaphores without S.L cioè senza queue/set
 - semaphore operations are still atomic, so there is no interleaving between the two statements implementing the wait(S) operation

```
wait(S) =
< await(S.V > 0)
  S.V ← S.V - 1 >
```

```
signal(S) =
< S.V ← S.V + 1 >
```

Si occupa solo di incrementare il valore, non di risvegliare i processi sospesi perché non ci sono

il processo non viene mai sospeso: controlla di continuo che S.V sia positivo (appena diventa positivo, il processo va avanti e decrementa S.V)

- Loosing freedom from starvation In questa situazione, c'è almeno uno scenario in cui un processo non acceda mai in CS
 - with busy-wait semaphores you cannot assume that a process enters in its critical section event in the 2-process solution
- Busy-wait semaphores are appropriate in a multiprocessor system when the waiting process has its own processor and is not wasting CPU time that could be used for other computation
 - they would be appropriate in a system with a little contention so that the waiting process would not waste too much CPU time

Ha senso applicare il Busy-Wait Semaphores in contesti in cui si perderebbe più tempo a fare Context Switch tramite Sistema Operativo (cioè sospendere il processo) che andare in await e quindi rimanere attivo controllando di continuo che una condizione sia vera (Usato in Architetture/Algoritmi molto parallele)

SEMAPHORE USAGE

I semafori sono dei meccanismi di basso livello

- Semaphores are primitive constructs that can be used as low-level building block to solve almost *any* problem concerning process interaction
 - in shared memory architecture
- In particular they can be used for both:
 - **mutual exclusion** (Competizione)
 - e.g. critical section problem
 - implementing *locks*
 - ...
 - **synchronisation** (Cooperazione)
 - event semaphore for signalling
 - barriers
 - ...

CRITICAL SECTION WITH SEMAPHORES

Usare semafori per implementare la sezione critica (usarli come lock)

- Using a semaphore, the solution of the critical section problem for two processes is trivial
 - using a semaphore as a lock

CS with semaphores: 2 processes	
binary semaphore $S \leftarrow (1, \{\})$ Inizializzo ad 1 il valore del semaforo (1 permesso: un processo alla volta entra nella propria sezione critica)	
P	Q
loop forever p1: NCS p2: wait(S) Pre-Protocol di P p3: CS p4: signal(S) Post-Protocol di P	loop forever q1: NCS q2: wait(S) Pre-Protocol di Q q3: CS q4: signal(S) Post-Protocol di Q

PROVING CORRECTNESS

- Building the reduced state diagram and checking properties (rimosse le CS e NCS)

CS with semaphores: 2 processes (abbreviated)	
binary semaphore $S \leftarrow (1, \{\})$	
p	q
loop forever p1: wait(S) Pre-Protocol di p p2: signal(S) Post-Protocol di p	loop forever q1: wait(S) Pre-Protocol di q q2: signal(S) Post-Protocol di q

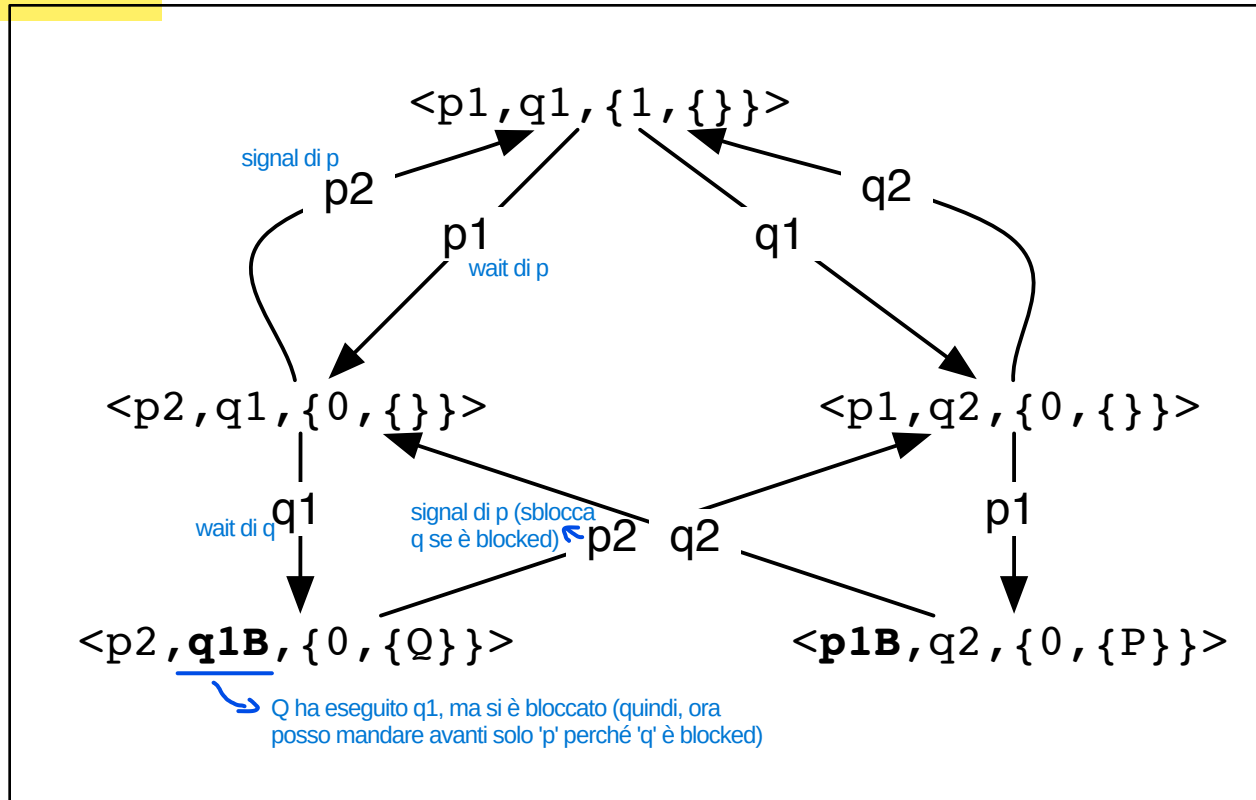
- It can be verified that the semaphore solution for the CS problem is correct
 - there is mutual exclusion, free from deadlock and starvation

sempre vero in ogni scenario se due processi (se avessi 3+ processi per avere starvation dovrei usare una lista/queue al posto del set)

"always not (p2 AND q2)": è sempre true in ogni scenario

STATE DIAGRAM WITH BLOCKED STATES

LTS



- Remarks

- semaphore structure in the tuple state $\{ s.V, s.L \}$
- *blocked state* for P and Q labelled as **p1B** and **q1B**

CRITICAL SECTION FOR N PROCESSES

- The same solution applies also for N processes

CS with semaphores: N processes

binary semaphore $S \leftarrow (1, \{\})$

Any process

```
loop forever
p1: NCS
p2: wait(S) Pre-Protocol
p3: CS
p4: signal(S) Post-Protocol
```

- But it there is no more freedom from starvation se si usa il set (se usassi liste/queue avrei ancora freedom from starvation)

USING SEMAPHORES FOR SYNCHRONIZATION

Più processi fanno la wait su stesso semaforo
 - Soluzione 1: il processo che chiama la signal la chiama più volte in base a quanti processi interessa quell'evento (es: 2 processi in wait su S1, allora sort1 chiama 2 signal). Se lo facessi solo una volta, avrei starvation (uno dei due processi in wait rimarrebbe bloccato).
 - Soluzione 2: creo N semafori in base a quanti processi interessa l'evento e il processo chiama la signal su N semafori (questa soluzione è concettualmente sbagliata: sto dando un ordine su quale semaforo chiamare prima)

- Semaphores provide a basic mechanism also to synchronise processes

- solving order of execution problems

> event semaphores

I semafori, utilizzati in questo modo, è come se tenessero traccia di quello che è capitato (hanno uno stato interno)

- used to send / receive a temporal signal

- initialised to (0, {})

- An example: merge sort

Essendo che il valore del semaforo ha solo un lower bound e non un upper bound, possiamo andare in contro ad un bug (esempio: un processo 3 fa due signal su un semaforo e due processi hanno await nelle proprie sequenze di istruzioni dove si ha un ciclo: il processo 2 è talmente veloce, rispetto al primo, che anziché fare await una sola volta, riesce a farlo due volte, grazie al ciclo che gli permette di ritornare sul await, quindi il processo 1 va in starvation perchè il processo 2 ha preso i due signal del processo 3). Un' altro problema nasce se si usano i set al posto delle liste/code.

Per risolvere, mettere in campo eventi specifici, quindi semafori distinti così da evitare di permettere di "fregare" l'evento dell'altro processo

Se prima di andare in wait, l'evento è già accaduto, il processo che ha chiamato la wait non si ferma, va avanti nella sua esecuzione

Più semafori ad eventi che rappresentano più eventi

Merge sort

binary semaphore S1 $\leftarrow (0, \{\})$ Prima parte è stata ordinata
 Semaforo evento inizializzato a 0 che indica che evento 1 ancora non è avvenuto
 binary semaphore S2 $\leftarrow (0, \{\})$ Seconda parte è stata ordinata
 Semaforo evento inizializzato a 0 che indica che evento 2 ancora non è avvenuto
 integer array A vettore da ordinare

sort1

p1: sort 1st half of A
 p2: signal(S1) segnalo a merge che ho terminato (evento 1)

sort2

q1: sort 2nd half of A
 q2: signal(S2) segnalo a merge che ho terminato (evento 2)

merge

r1: wait(S1) aspetto che evento avvenga
 r2: wait(S2) aspetto che evento avvenga
 r3: merge halves of A

THE PRODUCER-CONSUMER (P/C) PROBLEM

Rappresenta, in generale, il caso in cui si hanno due insiemi di processi costituiti ognuno da uno o più produttori/consumatori, dove i produttori generano dei dati per i consumatori (produttori e consumatori possono avere velocità diverse)

- The producer-consumer problem is an example of an order-of-execution problem
(esempio: i produttori lavano i piatti e li passano ai consumatori che li asciugano) (Se i produttori vanno a velocità più alta dei consumatori bisogna introdurre un buffer per evitare che il produttore si blocca aspettando il consumatore che finisca di processare i dati precedentemente passati: esempio del produttore che ha lavato il nuovo piatto, ma si blocca perchè il consumatore sta asciugando ancora quello precedente. Usando il buffer, si evita che il produttore si blocca perchè mette il dato creato in una struttura dati tramite cui il consumatore prende i dati prodotto dal produttore senza chiederli direttamente ad essi)
- Two types of processes:
 - **producers**
 - a producer process executes a statement, *produce*, to create a data element and then sends this element to the consumer process
 - **consumers**
 - ^{dopo aver ricevuto un} upon receipt of a data element from a producer process, a consumer process executes a statement, *consume*, with the data element as a parameter
- Ubiquitous patterns in CS:

PRODUCER	CONSUMER
Communication line	Web browser
Web browser	Communication line
Keyboard	Operating Systems
Word processor	Printer
Game program	Display screen
Proces ...	...

P/C WITH A BUFFER

- When a data element must be sent from one process to another, the communication can be
 - **synchronous**
 - communication cannot take place until both the producer and consumer are ready to do so
 - **asynchronous**
 - the communications channel itself has some capacity for storing data elements
 - disaccoppiamento uncoupling very useful ~~useful~~ for dynamic / open systems
 - temporal uncoupling among participants
 - dynamic set of processes
 - useful also when producers and consumers have different speed
- The asynchronous case needs the introduction of a proper **buffer** where to store and retrieve data
 - shared data structures with a mutable state, read by consumers and written by producers

1

P/C + INFINITE BUFFER

Il produttore, se è più veloce del consumatore, può riempire il buffer anche all'infinito (non ci sono limiti di memoria del buffer)

- If there is an infinite buffer, there is only one interaction that must be synchronised
 - the consumer must not attempt a take operation from an empty buffer

P/C with infinite buffer

Qua metto ciò che viene prodotto

UnboundedQueue<Item> buffer ← empty queue

semaphore availItems ← (0, { })

il valore del semaforo rappresenta il numero di risorse disponibili nel Buffer che possono essere elaborate dal Consumer, prodotte dal Producer

producer

loop forever

p1: Item el ← produce

Azione che produce il suo lavoro (concorrente con il resto)

p2: append(buffer, el)

p3: signal(nAvailItems)

Semaforo risorse: rappresenta il numero di risorse disponibile da essere prese da consumatore (la signal notifica che c'è un nuovo elemento sul buffer)

consumer

loop forever

q1: wait(availItems)

aspetta che ci siano degli elementi sul buffer (se 0, si blocca)

q2: Item el ← take(buffer)

q3: consume(el)

Quando si sblocca prendere l'elemento dal buffer ed esegue le operazioni di consume (in modo concorrente)

- invariant: `nAvailItems.V = #buffer`
 - actually true only if `p2+p3` and `q1+q2` are considered atomic
- Note that in this example *append* and *take* are meant to be atomic
- `nAvailItems` is called **resource semaphore**

Se non lo fossero, dovrei stare attento alle CS: bisogna mettere un Mutex sul Buffer se il Buffer è una struttura dati non atomica e condivisa

2 P/C + BOUNDED BUFFER

Viene aggiunto un limite sulla capacità del Buffer
(il produttore si blocca se il Buffer è pieno)

- In this case, there is also another interaction that must be synchronized
 - the producer must not attempt an append operation on a buffer which is full

P/C with *bounded* buffer

BoundedQueue<Item> buffer ← empty queue

semaphore availItems ← (0, {})

semaphore availPlaces ← (N, {}) semaforo che rappresenta quanti posti sono disponibili nel Buffer (N è il buffer size)

producer

loop forever

p1: Item el ← produce (concorrente)

p2: wait(availPlaces) se buffer pieno, si blocca; se no, continua e lo mette nel buffer

p2: append(buffer, el)

p3: signal(availItems)

consumer

loop forever

q1: wait(availItems)

q2: Item el ← take(buffer)

q3: signal(availPlaces)

q4: consume(el) (concorrente)

Quando il consumatore passa la wait, prende un elemento dal buffer e segnala che il buffer si è liberato di un elemento

- nAvailItems and nAvailPlaces are an example of **split semaphores**

- invariant: nAvailItems + nAvailPlaces = N

numero posti occupati nel buffer

numero posti liberi nel buffer

buffer size

Questo perchè la loro somma è uguale alla quantità totale di risorse disponibili

COMBINING MUTEX+SYNCH SEMAPHORES

Oltre a quello visto prima, Si introduce un altro semaforo come mutex per proteggere l'accesso alla struttura dati condivisa, quindi al Buffer, per evitare corse critiche

- Nella realtà, l'accesso al Buffer potrebbe non essere fatto in modo atomico di base: tipicamente il buffer è una struttura dati non Thread Safe
- As a generalisation of previous case, we consider the shared use of a *non-atomic* data structure (a buffer in this case), so with non-atomic operations
 - introducing a mutex for guaranteeing also mutual exclusion

P/C with *bounded* buffer with multiple producers & consumers

```
BoundedQueue<Item> buffer ← empty queue  
semaphore availItems ← (0, { })  
semaphore availPlaces ← (N, { })  
binary semaphore mutex ← (1, { })
```

l'accesso in sezione critica può avvenire contemporaneamente tra produttore e produttore, produttore e consumatore, consumatore e consumatore

Prima di fare append e take, bisogna prendere prima la Mutex per garantire che non ci siano corse critiche in questo accesso

producer

```
loop forever  
p1: Item el ← produce  
p2: wait(availPlaces)  
p3: wait(mutex) Pre-Protocol CS  
p4: append(buffer, el)  
p5: signal(mutex) Post-Protocol CS  
p3: signal(availItems)
```

consumer

```
loop forever  
q1: wait(availItems)  
q2: wait(mutex) Pre-Protocol CS  
q3: Item el ← take(buffer)  
q4: signal(mutex) Post-Protocol CS  
q4: signal(availPlaces)  
p4: consume(el)
```

DINING PHILOSOPHERS

- Classical problem in the field of concurrent programming
 - originated by an examination question set by Dijkstra in 1971 on a synchronisation problem where five computers competed for access to five shared tape drive peripherals
 - retold as the dining philosophers problem by Tony Hoare.
 - nowadays it is an entertaining vehicle for comparing various formalism for writing and proving concurrent problems
 - sufficiently simple & challenging
- Description
 - there is a secluded community of five philosophers who engage in only two activities: *thinking* and *eating*
 - (non per forza 5: N filosofi)
 - Fase in cui computano (Think and Eat) e Fase in cui accedono a coppie di risorse
 - meals are taken ~~communally~~ ^{in comune} at a table set with 5 plates and 5 forks
 - Coppie di risorse ← dividive
 - in the centre of the table there is a bowl of spaghetti that is endlessly replenished.
 - the spaghetti is hopelessly tangled and a philosopher needs *two* forks in order to eat
 - each philosopher may pick up the forks on his left and on his right, *but only one fork at a time*

DP PROPERTIES

Philosopher

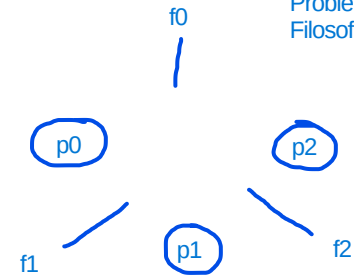
```
loop forever
p1: think
p2: <pre-protocol>
p3: eat
p4: <post-protocol>
```

- The problem is to design pre- and post- protocols to ensure the following properties:
 - a philosopher can eat^(eat: significa lavorare con le risorse) only if he/she has two forks^(per fare eat bisogna avere entrambe le risorse)
 - **mutual exclusion**^{le risorse possono essere utilizzate solo in modo mutuamente esclusivo (una risorsa non può essere utilizzate concorrentemente da più filosofi): una forchetta può essere utilizzata da un solo filosofo alla volta}
 - no two philosophers may hold the same fork simultaneously
 - **freedom from deadlock**
 - **freedom from starvation**
 - efficient behavior in the absence of contention

FIRST ATTEMPT

Vettore di Mutex su ogni risorsa

Problema dei
Filosofi con $N=3$



- Each fork is modelled as a semaphore

- wait => taking a fork Mi blocco se la risorsa non è disponibile;
avanzo/sblocco se la risorsa è disponibile
- signal => putting down the fork Dopo essere uscito da
CS, rilascio la risorsa

Dining philosophers (first attempt)

```
semaphore array[0..4] fork ← [1,1,1,1,1]
```

```
loop forever
```

```
p1: think NCS
```

```
p2: wait(fork[i])
```

```
p3: wait(fork[(i+1)%N])
```

```
p3: eat CS (computazione usando le risorse prese)
```

```
p4: signal(fork[i])
```

```
p5: signal(fork[(i+1)%N])
```

Vengono richieste le
forchette di Destra e Sinistra
Pre-Protocol CS

Vengono rilasciate le
forchette (risorse)
Post-Protocol CS

Se avessi un filosofo in
meno, non avrei il
problema di Deadlock

Avviene Deadlock perchè l'ultimo
filosofo continua a prendere la
forchetta di sinistra e poi quella di
destra e non prima la forchetta
con indice più piccola tra le due
forchette (cosa che viene fatto
dagli altri filosofi). Si potrebbe
risolvere implementando la
logica di prendere prima la
forchetta con indice più piccolo
(sarebbe la soluzione a PAG 31)

→ Soddisfa Mutua Esclusione

- It can be proved that no fork is ever held by two philosophers

- Unfortunately this solution **deadlocks**

Scenario in cui ogni filosofo ha la forchetta di sinistra e
si mettono in attesa di avere la forchetta di destra

- under an interleaving that has all philosophers pick up their left
forks before any of them tries to pick up the right fork

DEADLOCKS

- A situation ^{in cui} ~~wherein~~ two or more competing actions are waiting for the other to finish, and thus neither ever does
- Coffman *necessary conditions* for a deadlock to occur (1971) Condizioni che ci devono essere per avere deadlock
 - **mutual exclusion condition**
 - a resource that cannot be used by more than one process at a time
 - **hold and wait condition** Ogni processo ha bisogno di 2+ risorse
 - processes already holding resources may request new resources
 - **no preemption condition** Entrati in deadlock, non si possono liberare le risorse prese
 - no resource can be removed from a process holding it
 - resources can be released only by the explicit action of the process
 - **circular wait condition** Risorse condivise in modo circolare
 - two or more processes form a circular chain where each process waits for a resource that the next process in the chain holds
- Deadlock can only occur in systems where all 4 conditions hold true

DEADLOCKS WITH LOCKS

- It happens when
 - multiple threads wait forever due to cyclic locking dependency
 - simplest case
 - when thread A holds lock L and tries to acquire lock M, but at the same time thread B holds M and tries to acquire L, both thread will wait for ever
 - *deadly embrace* (i.e. deadlock)

DEADLOCKS DETECTION & RECOVERY

- Deadlocks **detection** and **recovery**
 - adopted in databases
 - databases are designed to detect and recover from deadlocks
 - transactions typically acquire many locks, until they commit
 - not so uncommon for two transactions to deadlock
 - identifying the set of transactions that are deadlocked by analysing *is-waiting* dependency graph
 - looking for cycles
 - if a cycle is found, a victim is selected and the transaction aborted
- No automated deadlock detection / recovery mechanism in JVM
 - if threads deadlock, *that's all, folks!*
 - we can just shutdown the application
 - “post-mortem” diagnosis support

In Java, si può rilevare
Deadlock, ma non si possono
usare meccanismi di recovery



BACK TO DP - FIRST SOLUTION:

TICKETS

Quando non è possibile stabilire un ordine dei lock, viene usata questa soluzione

2

- To ensure liveness we can limit the number of philosophers eating simultaneously (or entering the dining room)
 - introducing *meal* (or *room*) *tickets*
 - N-1 tickets for N philosophers
- Così si ha freedom from deadlock perchè contemporaneamente possono mangiare massimo N-1 filosofi (si rompe la catena wait-for)

Dining philosophers (solution)

```
semaphore array[0..4] fork ← [1,1,1,1,1]
semaphore ticket ← (4, { })
```

valore massimo è N-1; valore minimo è 0

```
loop forever
```

```
p1: think NCS
```

```
p2: wait(ticket)
```

```
p3: wait(fork[i])
```

```
p4: wait(fork[(i+1)%N])
```

```
p5: eat CS
```

```
p6: signal(fork[i])
```

```
p7: signal(fork[(i+1)%N])
```

```
p8: signal(ticket)
```

Pre-Protocol

Post-Protocol

Usando semaforo ticket: prima di prendere entrambe le forchette, si prende un ticket

- It can be proved that this solution satisfies all the properties (the ones in Slide 24)

2ND SOLUTION: BREAKING THE WAIT-FOR CHAIN

Vengono ordinati i lock e Vengono presi sempre nello stesso ordine (non applicabile in contesti in cui il numero dei lock può essere dinamico/variabile)

3

- It can be observed that there is no more deadlock if the last philosopher picks up first the right fork and then left one
 - breaking the wait-for chain
- Remark
 - actually - given the total order among the identifiers of the forks - the last philosopher was picking the forks in the opposite order with respect to the other philosophers
 - first the (N-1) fork and then the 0 fork
 - the solution is then about picking the forks always in the same order (dal identificatore più piccolo al più grande)

2ND SOLUTION: CODE

Dining philosophers (2nd solution)

```
semaphore array[0..4] fork  $\leftarrow$  [1,1,1,1,1]
```

```
integer first = min(i, (i+1)%N)  
integer second = max(i, (i+1)%N)
```

le forchette, per
quel filosofo, sono
sempre le stesse

```
loop forever  
p1: think  
p2: wait(fork[first])  
p3: wait(fork[second])  
p4: eat  
p5: signal(fork[first])  
p6: signal(fork[second])
```

GENERAL RULE FOR DEADLOCK AVOIDANCE

- General setting
 - N processes sharing and acquiring multiple locks
- General rules to avoid deadlock
 1. assign a total order to locks
 2. acquire the locks *always in the same order*
- It works because it makes it impossible to have circular wait-for dependency among processes
 - necessary condition for deadlock

READERS-WRITERS PROBLEM

- The problem of readers-writers is similar to the mutual exclusion problem in that several processes are competing for access to a critical section [Courtois, Heymans, Parnas - 1971].
- In this problem, however, we divide the processes into two classes:
 - **Readers**
 - which are required to exclude writers but not other readers
 - **Writers**
 - which are required to exclude both readers and other writers
- The problem is an abstraction of *access to databases* (or any kind of shared resource)
 - no danger in having process reading data concurrently
 - writing or modifying data must be done under mutual exclusion to ensure consistency of the data
- Solutions must satisfy these **invariants**

A $nR \geq 0$ numero lettori può essere, all'interno della CS, maggiore o uguale a 0

B $nW = 0 \mid nW = 1$ numero scrittori in CS uguale a 0 o 1

C $(nR > 0 \rightarrow nW = 0) \wedge (nW = 1 \rightarrow nR = 0)$  Bisogna evitare corse critiche
Se ho anche solo un lettore in CS, devo avere 0 scrittori in CS
Se ho uno scrittore in CS, devo avere 0 lettori in CS

nR = number of readers, nW = number of writers

1

AN OVER-CONSTRAINED SOLUTION

- Using a single semaphore functioning as a *lock*

Readers-and-writers: first attempt	
<pre>binary semaphore rw ← (1,{}) DataBase dbase;</pre>	
reader	writer
<pre>loop forever p1: wait(rw) p2: Item el ← read(dbase) p3: signal(rw)</pre>	<pre>loop forever q1: wait(rw) q2: Item el ← create_record; q3: write(dbase,el) q4: signal(rw)</pre>

- Each reader and writer has exclusive access to the dbase
- over-constrained** solution: serialising access also for readers!

↳ perchè non devo bloccare l'accesso in Sezione Critica quando più processi lettori vogliono leggere

Idea: il mutex rw bisogna averlo per l'accesso a CS, ma contiamo il numero dei lettori che sono dentro la CS

I reader in entrare, se rw è già stato preso da un reader precedente, non competono mai con il writer in wait su rw: ci possono essere dei raffinamenti di questo soluzione per evitare che i writer vada in starvation

- Readers don't use the same lock of writers
 - mutexR lock for readers for updating common data structures (nr)

Readers-and-writers: solution

```
binary semaphore mutexR ← (1, { }) semaforo per accedere alla CS dei reader per controllare, incrementare e decrementare nr
int nr ← 0
binary semaphore rw ← (1, { }) semaforo mutex per accedere alla CS reader/writer per leggere/scrivere sul dbase
DataBase dbase;
```

mutexR utilizzata per fare check su una variabile condivisa ed incrementarla/decrementarla evitando corse critiche (CS rispetto ai reader)

reader

Serve per evitare corse critiche tra i lettori, dato che devo incrementare/decrementare il numero di lettori (parte mutuamente esclusiva tra reader)

```
loop forever
p1: wait(mutexR)
p2: if (nr == 0)
p3:   wait(rw)
p4:   nr ← nr + 1
p5:   signal(mutexR)
p6:   Item el ← read(dbase)
p7:   wait(mutexR)
p8:   nr ← nr - 1
p9:   if (nr == 0)
p10:    signal(rw)
p11: signal(mutexR)
```

Se sono il primo lettore cerco di prendere il mutex rw, quindi vuol dire che sono l'unico lettore in CS (se c'è già uno scrittore in CS, mi blocco; se non c'è, prendo il mutex rw e non mi blocco). Se non sono il primo, vuol dire il mutex rw l'ha già preso qualcun altro reader e posso entrare in CS

CS
reader
/writer

Arrivato in p4, significa che posso accedere a CS rispetto a reader/writer

Se sono ultimo reader ad uscire, devo rilasciare anche il mutex rw

writer

```
loop forever
q1: wait(rw) Pre-Protocol CS reader/writer
q2: Item el ← create_record;
q3: write(dbase, el)
q4: signal(rw) Post-Protocol CS reader/writer
```

read e write sono le operazioni più pesanti (read può essere fatto in interleaved tra readers)

OTHER PROBLEMS

- “Barber Shop” Problem (Dijkstra)
- “Cigarette Smokers” Problem (Patil, 1971)
- ...
- “The Santa Claus” problem (Bansal, 2006)

THE CIGARETTE SMOKER'S PROBLEM

- Synchronisation problem proposed by S. Patil in 1971, to investigate the limits of the semaphore primitive
- Problem statement
 - assume that there is a group of four people: 3 *smokers* and 1 *agent (arbiter)*. To roll and smoke a cigarette three ingredients are needed: paper, tobacco, ~~matches~~^{fiammiferi}. One of the smokers has an infinite supply of papers, another has an infinite supply of tobacco, and another has an infinite supply of ~~matches~~^{fiammiferi}. The agent has an infinite supply of all three ingredients.
 - the four participants repeatedly perform the following: the agent puts two ingredients on the table; the smoker who has the remaining ingredient takes the two ingredients, rolls a cigarette, smokes it, and notifies the agent on completion. Then the agent puts another two ingredients on the table, and so on
 - the problem is to write a program to synchronize the agent and the smokers

PATIL'S ARGUMENT

Perché i Semafori "Falliscono" qui?
Il problema fondamentale identificato da Patil è la mancanza di atomicità nel testare più condizioni.
Se un fumatore ha bisogno di Tabacco (S[1]) e Carta (S[2]), il suo codice ideale sarebbe:
wait(S[1] AND S[2])
Tuttavia, i semafori di Dijkstra permettono solo di aspettare una risorsa alla volta:
wait(S[1])
wait(S[2])

- Patil's argument was that Edsger Dijkstra's semaphore primitives were limited
 - he used the cigarette smokers problem to illustrate this point by saying that it cannot be solved with ^{only} semaphores.
- However, Patil placed heavy constraints on his argument:
 - the process code is the following ^(codice dell'Agente) (and is not modifiable)

```
shared S: array[1..3] of binary semaphores, initially all 0 array delle 3 risorse
  agent: binary semaphore, initially 1
local i, j: range over [1,2,3]
loop
  set i and j (at random) to two different values from [1,2,3]
  wait(agent)
  signal(S[i])
  signal(S[j])
end_loop
```

Senza istruzioni condizionali (if/else), non possiamo creare una logica che dica: "Se è arrivato il tabacco, controlla se c'è anche la carta; se no, aspetta".
Se usiamo la sequenza wait(S[1]) seguita da wait(S[2]), rischiamo il Deadlock:
- L'Agente mette sul tavolo Tabacco e Fiammiferi.
- Il Fumatore che ha bisogno di Tabacco e Carta consuma il semaforo del Tabacco e si blocca aspettando la Carta (che non arriverà).
- Il Fumatore che ha bisogno di Fiammiferi e Carta (e che potrebbe procedere se avesse il fiammifero) non può farlo perché il "pezzo" mancante è bloccato dall'altro fumatore.

- the solution is not allowed to use conditional statements or an array of semaphores.

With these two constraints, a semaphore-based solution to the cigarette smokers problem is impossible.

Secondo Patil, i semafori sono una primitiva troppo povera perché non permettono a un processo di decidere cosa fare in base a quale risorsa è disponibile senza "impegnarsi" (consumando il semaforo).
Nota storica: In seguito, si è dimostrato che il problema è risolvibile usando dei processi intermediari (i "Pusher") che non fumano ma coordinano i segnali, ma Patil sostenebbe che questo "aggira" il limite aggiungendo complessità che la primitiva stessa dovrebbe gestire.

BEYOND SEMAPHORES...

- Semaphores are a powerful construct, but very low level
 - error-prone programs
 - hard to use in complex concurrent programs
- > looking for high-level constructs: **monitors**
 - introduced by Brinch Hansen (1973)
 - generalised by Hoare (1974)

MONITORS

Immagina un Monitor come una "stanza protetta" che contiene:

- I Dati (Stato): Le variabili condivise (es. un contatore, una coda, il tavolo dei fumatori).

- Le Operazioni: Metodi o funzioni per manipolare quei dati.

- La Politica di Concorrenza: Un meccanismo automatico che garantisce la Mutua Esclusione.

Solo un processo alla volta può essere "dentro" il monitor. Se un secondo thread prova a chiamare un metodo mentre il primo è ancora dentro, il secondo viene messo automaticamente in attesa in una Entry Queue.

- def. **Monitor**

- a concurrent programming data abstraction ^{che incapsula} encapsulating the ~~synchronisation and mutual exclusion policy in accessing it~~ _{la politica di sincronizzazione e di mutua esclusione per l'accesso alle risorse che contiene.}

- state + operations + concurrency policy
- like a *module* + basic mechanisms to enforce correctness in module concurrent access

- Generalisation of the *kernel* or *supervisor* concept in operating systems, where critical sections such as the allocation of memory are centralised in a privileged program

- applications programs request services which are performed by the kernel
- kernels are run in a HW mode that ensures that they cannot be interfered with by application programs
- monitors as decentralised versions of the monolithic kernel

- Generalisation of the *object* notion in OOP

È essenzialmente una Classe "Thread-Safe"

- classes encapsulating data + operation + synchronisation / mutex policy

- Incapsulamento: I dati sono privati.

- Interfaccia: Si accede ai dati solo tramite metodi public.

- Sincronizzazione: In Java, ad esempio, basta aggiungere la keyword `synchronized` a un metodo per trasformare quell'oggetto in un monitor.

Il monitor è un'applicazione di questa idea a livello di software. Invece di avere un unico grande Kernel che gestisce tutto, hai tanti piccoli "micro-kernel" (i Monitor) distribuiti nel tuo programma, ognuno responsabile di una specifica risorsa.

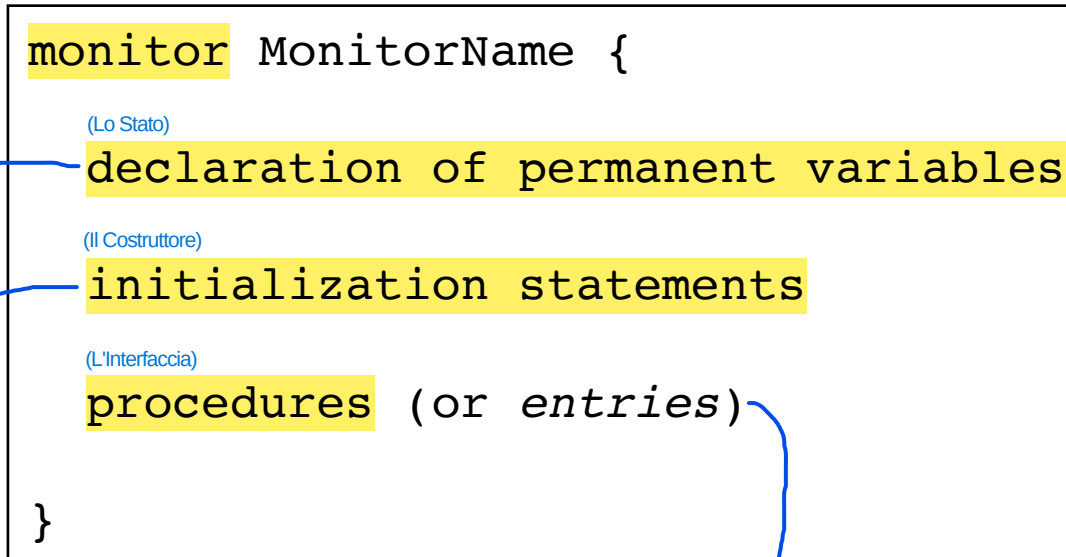
MONITOR DEFINITION

- Monitor are declared and created in different ways according to the specific language
- An abstract representation:

Queste sono le variabili "private" del monitor.

- Permanenti: Esistono per tutta la vita del monitor.
- Protette: Non possono essere toccate dall'esterno. Solo le procedure interne al monitor possono leggerle o modificarle.

Codice eseguito una sola volta al momento della creazione del monitor.



Sono i "punti di ingresso" per i processi esterni.

- Mutua Esclusione Automatica: Quando un processo chiama una procedure, il sistema controlla se c'è già qualcun altro nel monitor.
> Se il monitor è libero, il processo entra.
> Se il monitor è occupato, il processo viene messo in una Entry Queue e sospeso.

MONITOR PROPERTIES

- Monitors as instances of abstract data type
 - *only operations (procedures) name are visible outside the monitor*
 - they are the *interface*
 - they provide the only gates through the “wall” defined by the monitor declaration
 - call to monitor procedures:
`call MonitorName.OpName ( params )`
(often written simply `MonitorName.OpName ( params )`)
 - statements within the monitor cannot access variables declared outside ~~the~~ ^{the} monitor
 - permanent variables are initialised before any procedure is called

1 MONITOR FEATURES: MUTUAL EXCLUSION

- **Intrinsic / implicit mutual exclusion**
 - procedures *by definition* execute with mutual exclusion
 - a monitor procedure is called by an external process
 - a procedure is active if some process is executing a statement in the procedure
 - *at most one instance of one monitor procedure may be active at a time* può essere attiva al massimo una sola procedura alla volta di un monitor
 - processes that find the monitor 'busy' are suspended

SIMPLE EXAMPLE

- Classic counter
 - but thread-safe, thanks to monitor properties

```
monitor Counter {  
  
    int count; Variabile permanente  
  
    {  
        procedure inc(){  
            count := count + 1  
        }  
        procedure getValue():int {  
            return count;  
        }  
    }  
}
```

Procedure che lavorano sulla Variabile permanente ←

REMARKS

Chi "comanda" il lucchetto?

- Nei Semafori: il programmatore deve ricordarsi di scrivere wait(mutex) prima di entrare in una sezione critica e signal(mutex) quando esci. Se ti dimentichi una riga, il sistema fallisce.
- Nel Monitor: Tu scrivi solo il codice della procedura. È il compilatore o il supporto a tempo di esecuzione (run-time) che aggiunge automaticamente il "lucchetto". Tu non vedi il codice che gestisce il blocco, succede "dietro le quinte".

- The mutual exclusion is **implicit** and does not require the programmers to use any other mechanism (such as wait and signal..)
 - if operations of the same monitor are called by more than one process, the implementation ensures that these are executed under mutual exclusion
 - > operations are executed **atomically** (with respect to each other)
 - if operations of different monitors are called, their execution can be interleaved
- There is no explicit queue associated with the monitor procedure
 - *starvation* problem

Nelle implementazioni teoriche più pure dei monitor, la specifica non impone che la Entry Queue sia una coda FIFO (First-In, First-Out).

2

MONITOR FEATURES: SYNCHRONIZATION

- Entry Queue: Dove finisci perché il monitor è occupato da un altro processo. È gestita dal Monitor stesso.
- Condition Queue: Dove finisci perché, anche se sei entrato, mancano i dati per procedere. È gestita dal programmatore tramite le condition variables.

- **Explicit synchronisation support**
 - through **condition variables**
 - used inside the monitors by the programmers ^{per ritardare} to delay a process that cannot safely continue executing until the monitor's state satisfies some boolean condition
 - used also to awake a delayed process when the condition becomes true

Perché è "Esplicita"?

- Il programmatore deve dichiarare la variabile condizione (es. condition magazzinoPieno;).
- Il programmatore deve decidere quando chiamare wait (usando un if o un while).
- Il programmatore deve decidere quando chiamare signal per avvisare gli altri.

In sintesi: La mutua esclusione garantisce che non ci scontriamo; la sincronizzazione (condition variables) garantisce che collaboriamo nel momento giusto.

CONDITION VARIABLES

- Primitive data types that can be used to suspend (wait) ^{e riprendere l'esecuzione dei} and resume (signal) processes inside a monitor
 - ^{che rappresentano condizioni} ~~representing conditions~~ (events) on the monitor state ^{che attendono di} ~~that wait to~~ ^{essere soddisfatte e che vengono soddisfatte} ~~be satisfied and that becomes satisfied~~
- Two basic atomic operations, **waitC** and **signalC**
 - sometimes written simply wait and signal
- Each condition variable is associated with a FIFO queue of blocked processes

Questo significa che i processi vengono svegliati nell'ordine esatto in cui si sono messi in attesa.

- Se il Processo A va in wait alle 10:00 e il Processo B alle 10:05, la prossima signal sveglierà sicuramente il Processo A.
- Questo aiuta a prevenire il problema della Starvation all'interno delle condizioni (anche se, come abbiamo visto prima, la Starvation potrebbe ancora avvenire nella coda di ingresso generale del monitor).

Quando un processo esegue waitC(condizione):

- Rilascio immediato: Il processo "molla" il lucchetto del monitor (permette ad altri di entrare).
- Sospensione: Il processo viene sospeso e messo nella coda specifica di quella condizione.
- Stato di blocco: Rimarrà lì finché qualcuno non lo sveglia.

Quando un processo esegue signalC(condizione):

- Notifica: Controlla la coda associata a quella condizione.
- Ripristino: Se c'è almeno un processo in attesa, il primo della fila viene "svegliato".
- Nessun effetto: Se la coda è vuota, la signalC non fa nulla (a differenza dei semafori, non "accumula" memoria per il futuro).

COND. VARIABLE OPERATIONS

- **waitC(cond)**

- suspend the execution of the process and release lock of the monitor
- abstract implementation:

operazione atomica

```
waitC(cond) =  
< append p to cond.queue  
  p.state := blocked  
  monitor.lock := release >
```

- **signalC(cond)**

- unblock a process waiting on a condition
- abstract implementation:

operazione atomica

```
signalC(cond) =  
< if cond.queue != empty  
  q := remove head of cond.queue  
  q.state := ready >
```


SIMPLE SYNCH. EXAMPLE

- Synchronised cell

```
monitor SynchCell {  
  
    int value;  
    boolean available := false;  
    cond isAvail;  
  
    procedure set(int v){  
        value := v  
        available := true  
        signalC(isAvail)  
    }  
  
    procedure get():int {  
        if (!available)  
            waitC(isAvail)  
        return value  
    }  
}
```

REMARK

- There is an explicit link between condition variables and their encapsulating monitor

sulla variabile di condizione

wait operation releases the monitor lock

- This is essential to avoid that a process executing a waitC would block the access to the monitor

OTHER PRIMITIVES

- **emptyC(cond)** of a specific condition
 - check if the queue is empty
- **signalAllC(cond)** (sveglia tutti i processi in attesa sulla coda di quella condizione.)
 - like signal, but all the processes waiting on the condition are resumed
- **waitC(cond,rank)**
 - wait in order of increasing value of rank
- **minrank(cond)**
 - returns the value of rank of process at front of wait queue

Qui passiamo dalla semplice coda FIFO (First-In, First-Out) ad una Priority Queue.

- waitC(cond, rank): Invece di mettersi semplicemente in coda, il processo si inserisce in una posizione determinata dal valore rank. Più basso è il valore, più alta è la priorità.
- minrank(cond): Permette di sapere qual è il valore di priorità del processo che si trova in testa alla coda (quello che verrebbe svegliato per primo).

Perché sono utili?

Servono in situazioni dove l'ordine di arrivo non è importante quanto l'urgenza.

SYNCH CELL REVISITED /2

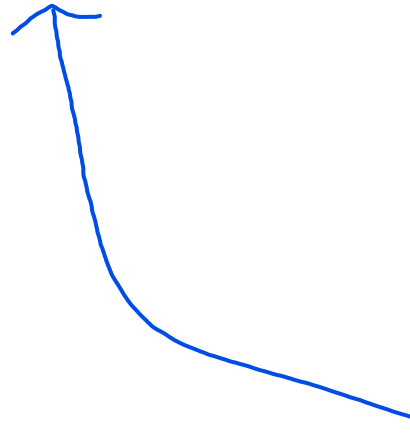
- Using signalAllC to wake up every process in the queue

```
monitor ImprovedSynchCell {  
  
    int value;  
    boolean available;  
    cond isAvail;  
  
    procedure set(int v){  
        value := v  
        available := true  
        signalAllC(isAvail)  
    }  
  
    procedure get():int {  
        if (!available)  
            waitC(isAvail)  
        return value  
    }  
}
```

SYNCH CELL REVISITED /3

- The same behavior can be obtained without signalAllC
 - signaling as soon as a suspended process is awoken

(anziché risvegliare tutti in una volta sola con signalAllC, si può fare un risveglio a catena: cioè quello che viene svegliato, sveglia il prossimo e così via)



```
monitor ImprovedSynchCell {  
  
    int value;  
    boolean available;  
    cond isAvail;  
  
    procedure set(int v){  
        value := v  
        available := true  
        signalC(isAvail)  
    }  
  
    procedure get():int {  
        if (!available){  
            waitC(isAvail)  
            signalC(isAvail)  
        }  
        return value  
    }  
}
```

IMPLEMENTING A SEMAPHORE

- Two implementations of a semaphore using monitors

```
monitor Semaphore {
  integer s := <InitValue>
  cond notZero

  procedure wait(){
    if s = 0
      waitC(notZero)
    s := s - 1
  }
  procedure signal(){
    s := s + 1
    signalC(notZero)
  }
}
```

```
monitor Semaphore {
  integer s := <InitValue>
  cond notZero

  procedure wait(){
    if s = 0
      waitC(notZero)
    else
      s := s - 1
  }
  procedure signal(){
    if emptyC(notZero)
      s := s + 1
    else
      signalC(notZero)
  }
}
```

SEMAPHORES VS. CONDITION VARIABLE IN MONITORS

SEMAPHORE	MONITOR
wait may or may not block wait non blocca sempre; se il valore del semaforo è >0, il processo prosegue senza fermarsi.	waitC always blocks L'operazione waitC(C) non valuta se una condizione logica sia vera o falsa; si limita ad addormentare il processo in modo incondizionato.
signal always has an effect	signalC has no effect if queue is empty
signal unblocks an arbitrary blocked process (dipende se è un set o FIFO queue)	signalC unblocks the process at the head of the queue
a process unblocked by signal can resume execution immediately	depending on the specific signaling semantics, a process unblocked by signalC must wait for the signaling process to leave the monitor

SIGNALING DISCIPLINE

- When a process executes a signal, even if there could be multiple processes ready to execute within the monitor, *only one process can have exclusive access*

i.e. because of the basic semantics of monitors

— only one process is chosen to keep active

Uno tra

→ either the signalling or the waiting process can be resumed, not both

- classic solution for monitors

Quando un processo esegue una signal, il sistema si trova improvvisamente con due processi idonei a entrare nel Monitor:

- Il Processo Segnalatore (che ha appena chiamato signal ed è ancora dentro).
- Il Processo in attesa (che è stato appena risvegliato dalla signal).

Poiché la regola d'oro è "al massimo un'istanza attiva", il Monitor deve decidere chi deve uscire o mettersi in attesa (varie strategie elencate nella prossima slide).

SIGNALING DISCIPLINE: SEMANTICS

A

Signal and Continue

È la politica più comune (usata in Java, Pthreads).

- the ~~signaller~~ continues and the ~~signalled~~ process executes at some later time
- nonpreemptive

Il problema della "Condizione Instabile": Poiché il Segnalatore continua a lavorare, tra il momento in cui viene inviato il segnale e il momento in cui il processo risvegliato riesce effettivamente a entrare nel monitor, qualcun altro potrebbe essere entrato e aver cambiato lo stato.

B

Signal and Wait

Qui il Segnalatore si ferma e il processo risvegliato parte subito.

- ~~signalled~~ process executes now and the ~~signaller~~ waits, eventually competing with other processes waiting for entering the monitor
- preemptive

La condizione che era vera al momento della signal è molto probabilmente ancora vera quando il processo risvegliato parte.

Il Segnalatore viene "prelevato" dal monitor e messo in attesa. Il Segnalatore torna a competere con tutti gli altri processi che vogliono entrare nel monitor per la prima volta.

C

Signal and Urgent Wait (or Immediate Resumption Requirement)

Questa è la soluzione classica

- like signal and wait, but the ~~signaller~~ has priority over processes waiting for the lock
- classic solution for monitors

Priorità assoluta: Quando il Segnalatore si ferma, non va a competere "alla pari" con gli altri in attesa fuori dalla porta. Viene messo in una Urgent Queue (coda urgente) che ha la precedenza su chiunque altro voglia entrare.

Vantaggio: È la variante che garantisce la massima stabilità della condizione logica. Il risveglio è immediato e protetto.

SIGNALING DISCIPLINE: SEMANTICS

- Given
 - S = precedence of the ~~signalling processes~~ processi segnalatori
 - W = precedence of the ~~waiting processes~~ processi in wait
 - E = precedence of processes blocked on an procedure

- Signalling processes: Rappresentano i processi che si trovano già attivi dentro il monitor e che, a un certo punto, eseguono un'istruzione di `signalC(cond)` per svegliare qualcun altro.

- Waiting / Signalled processes: I thread "svegliati". Sono i processi che si erano precedentemente messi in attesa su una variabile di condizione eseguendo una `waitC(cond)` e che sono stati appena svegliati dalla `signalC` di un segnalatore.

- Entry processes: I thread "esterni" in coda d'ingresso. Sono i processi che cercano di entrare nel monitor per la prima volta chiamando una delle sue procedure pubbliche, ma che hanno trovato il monitor occupato e sono rimasti bloccati nella Entry Queue.

A • Signal and Continue

- $E < W < S$

Qui la priorità è sbilanciata a favore di chi sta già lavorando.

- S (Signaller) è il re: Continua ad eseguire finché non ha finito.
- W (Wait) arriva dopo: Si mette in coda dopo il Segnalatore.
- E (Entry) sono gli ultimi: Chi sta fuori deve aspettare che sia S che W abbiano finito.

Nota: È la forma più "rilassata" perché non interrompe nessuno.

```
while (!B)
    wait(cond)
<access>
```

Il problema della "Condizione Instabile": Poiché il Segnalatore continua a lavorare, tra il momento in cui viene inviato il segnale e il momento in cui il processo risvegliato riesce effettivamente a entrare nel monitor, qualcun altro potrebbe essere entrato e aver cambiato lo stato. Per questo, È obbligatorio usare un `while` invece di un `if` quando si controlla la condizione

B • Signal and Wait

- $E = S < W$

```
if (!B)
    wait(cond)
<access>
```

Qui si cerca di dare una spinta al processo che è rimasto bloccato più a lungo.

- W (Wait) ha la priorità assoluta: Appena viene svegliato, il processo W ha la precedenza su tutti.
- $E=S$: Il segnalatore S perde la sua "protezione" speciale e viene trattato esattamente come un processo esterno E che sta bussando alla porta. Entrambi devono competere ad armi pari per rientrare.

C • Signal and Urgent Wait

- $E < S < W$

Questa è la disciplina più rigida e "giusta" dal punto di vista della logica di coordinazione.

- W (Wait) è il primo: Come sopra, ha priorità massima.
- S (Signaller) è il secondo: Non viene "declassato" al livello di E . Viene messo in una coda speciale (l'Urgent Queue) che gli garantisce di passare prima di chiunque stia arrivando dall'esterno (E).
- E (Entry) è l'ultimo: Chi arriva da fuori deve aspettare che il sistema interno si sia riassetato.

USING MONITORS

I Monitor sono lo strumento preferito per risolvere i problemi classici di concorrenza, proprio perché spostano la responsabilità della sincronizzazione dal programmatore alla struttura stessa del Monitor.

- Monitors can be used to implement any resource or data structure which is used concurrently by multiple processes and in which we want to encapsulate the synchronization policies
- Revisiting the main examples
 - *Producers-Consumers*
 - implementing the bounded-buffer as a monitor
 - *Readers-and-Writers*
 - implementing the rw-lock as a monitor
 - *Resource allocation and management*
 - implementing the resource allocator as a monitor

PRODUCERS-CONSUMERS

```
monitor BoundedBuffer {  
  
  ElemType buffer := <EmptyBuffer>  
  cond notFull, notEmpty;  
  
  procedure put(ElemType elem){  
    if (buffer is full)  
      waitC(notFull) aspetto che non sia più pieno  
    append(buffer, elem)  
    signalC(notEmpty) segnalo al consumatore che ho messo un elemento  
sul buffer (utile nel caso il buffer era vuoto prima)  
  }  
  
  procedure take(): ElemType {  
    if (buffer is empty)  
      waitC(notEmpty) aspetto che non sia più vuoto  
    ElemType el := head(buffer)  
    signalC(notFull) segnalo al consumatore che ho preso un elemento  
dal buffer (utile nel caso il buffer era pieno prima)  
    return el  
  }  
}
```

Producer	Consumer
loop p1: ElemType el := produce p2: BoundedBuffer.put(el)	loop q1: ElemType el := BoundedBuffer.take() q2: consume(el)

PRODUCERS-CONSUMERS

- Using a circular array for implementing the buffer data structure ...

```
monitor BoundedBuffer {

    int[] elems := new int[MAX_ELEMS]
    int first := 0, last := 0
    cond notFull, notEmpty

    procedure put(int elem){
        if ((last + 1) % MAX_ELEMS) = first
            waitC(notFull)
        elems[last] = elem
        last := (last + 1) % MAX_ELEMS
        signalC(notEmpty)
    }

    procedure take(): int {
        if (first = last)
            waitC(notEmpty)
        int elem = elems[first]
        first = (first + 1) % MAX_ELEMS
        signalC(notFull)
        return elem
    }
}
```

READERS-AND-WRITERS

(signal-and-continue)

```
monitor RWLock {  
    int nr, nw = 0;  
    cond okToRead, okToWrite;
```

```
    procedure void request_read(){
```

```
        while (nw > 0)  
            waitC(okToRead);
```

Aspetto che non ci siano più scrittori (uso il while perchè al momento del risveglio potrebbe essere che la condizione non sia più corretta, essendo S&C)

```
        nr := nr + 1;
```

```
    }
```

```
    procedure void release_read(){
```

```
        nr := nr - 1;
```

```
        if (nr = 0)
```

```
            signalC(okToWrite)
```

Se sono l'ultimo lettore, risveglio uno scrittore se in queue

```
    }
```

```
    procedure void request_write(){
```

```
        while (nr > 0 or nw > 0)  
            waitC(okToWrite)
```

Aspetto che non ci siano più lettori (uso il while perchè al momento del risveglio potrebbe essere che la condizione non sia più corretta, essendo S&C)

```
        nw := nw + 1;
```

```
    }
```

```
    procedure void release_write(){
```

```
        nw := nw - 1;
```

```
        signalC(okToWrite);
```

Risveglio un'altro scrittore e poi tutti gli altri lettori, se sono in queue

```
        signalAllC(okToRead);
```

```
    }
```

```
}
```

Invariant:

$(nr == 0 \text{ or } nw == 0) \text{ and } (nw \leq 1)$

```

monitor RWLock {
  integer readers := 0
  integer writers := 0
  cond okToRead,okToWrite;

  procedure startRead(){
    if writers != 0
      waitC(okToRead)
    readers := readers + 1
    signalC(okToRead)
  }
  procedure endRead(){
    readers := readers - 1
    if readers = 0
      signalC(okToWrite)
  }
  procedure startWrite(){
    if writers != 0 or readers != 0
      waitC(okToWrite)
    writers := writers + 1
  }
  procedure endWrite(){
    writers := writers - 1
    if emptyC(okToRead)
      then signalC(okToWrite)
      else signalC(okToRead)
  }
}

```

READERS-AND-WRITERS solution #2

Reader	Writer
p1: RWLock.startRead	q1: RWLock.startWrite
p2: read the dbase	q2: write the dbase
p3: RWLock.endRead	q3: RWLock.endWrite

RESOURCE ALLOCATION

Il monitor non si limita a proteggere una variabile (come nel buffer), ma diventa un arbitro decisionale che gestisce l'accesso a risorse limitate

- Monitors are typically used to function as **resource allocator**
 - enforcing some policy in allocating resources to processes requesting resource access

Il monitor mantiene internamente lo stato delle risorse:
- Variabili interne.
- Politiche (Policy): Il monitor incapsula la logica.

```
monitor ResourceAllocator {  
  
    procedure request(<Params>){  
        Ⓛ block the request  
        until the resource or a resource  
        is available, according to some policy Ⓡ  
    }  
  
    procedure release(<Params>){  
        Ⓛ possibly unblock some pending request Ⓡ  
    }  
}
```

request():
- Verifica la politica: "La risorsa è libera?" o "Il processo ha diritto a riceverla ora?".
- Se no: waitC(condizione_di_attesa). Il processo si sospende, liberando il lock del monitor.
- Se sì: Aggiorna lo stato (risorse_libere--) e procede.

release():
- Aggiorna lo stato: risorse_libere++.
- Decisione: Chi deve essere svegliato? signalC (o signalAllC) viene invocata per sbloccare i processi che erano in attesa.

MONITOR IMPLEMENTATION

- Monitor can be realised using semaphores, in particular

one semaphore `mutex` for mutual exclusion (Semaforo di Mutua Esclusione) Serve a garantire che solo un thread alla volta entri nel monitor.

for each condition variable (Semaforo per ogni condizione) a semaphore `condsem` and a counter È la "coda" di sospensione per chi aspetta che una condizione diventi vera.

`condcount` keeping track of the number of processes suspended on the variable

Tiene traccia di quanti thread sono bloccati su quella specifica condizione.

Signal and Continue semantics:

Prima di Prologue for each operation:

`wait(mutex)` Entrando, il processo deve acquisire il mutex. Se è già occupato da qualcun altro, si ferma qui (Entry Queue).

Dopo Epilogue for each operation:

`signal(mutex)` Uscendo, il processo rilascia il mutex. Questo è fondamentale: se non lo facessi, il monitor rimarrebbe bloccato per sempre dopo la prima chiamata

`waitC(cond) =`
`condcount++;`
`signal(mutex);`
`wait(condsem);` Il processo si mette in wait nella coda specifica della condizione.
`wait(mutex);` Quando il processo viene svegliato (da una `signalC`), non rientra subito nel monitor. Deve prima ri-acquisire il mutex. Solo dopo aver preso il lock, il processo può tornare a lavorare.

`signalC(cond) =`
`if (condcount > 0){`
`condcount--;`
`signal(condsem)`
`}`

È Signal-and-Continue perché, dopo la `signal` (`condsem`) dentro `signalC`, il processo segnalatore non si ferma. Non c'è alcun comando che lo costringe a cedere il mutex. Continua a eseguire le sue istruzioni fino a quando non arriva all'epilogo (`signal(mutex)`).

Signal and Wait semantics:

Prima di Prologue for each operation:

`wait(mutex)`

Dopo Epilogue for each operation:

`signal(mutex)`

`waitC(cond) =`
`condcount++;`
`signal(mutex);`
`wait(condsem);`

`signalC(cond) =`
`if (condcount > 0){`
`condcount--;`
`signal(condsem);`
`wait(mutex);`
`}`

Perché questa è "Signal-and-Wait"?
Qui, il processo segnalatore (che ha appena chiamato `signalC`) si auto-sospende eseguendo `wait(mutex)`.

BUILDING REUSABLE COORDINATION COMPONENTS AS MONITORS

- Exploiting monitors to realise reusable synchronisation / coordination components
 - latches
 - barriers
 - rendez-vous
 - message boxes
 - blackboards
 - event services
 - ...